HW5_Report

✏️ Author: 111511198 鄭恆安

Concept and Code Explanation of Naive Bayes Classifier

Naïve Bayes is a probabilistic classifier based on Bayes' theorem, assuming strong independence between features. The model predicts the class `C` for a sample `X` by maximizing the posterior probability $P(C|X)$, which can be expressed as:

$$P(C|X) \propto P(C) \prod_{i=1}^{n} P(X_i|C)$$

where $P(C)$ is the prior probability, and $P(X_i|C)$ is the likelihood of the feature X_i given the class C.

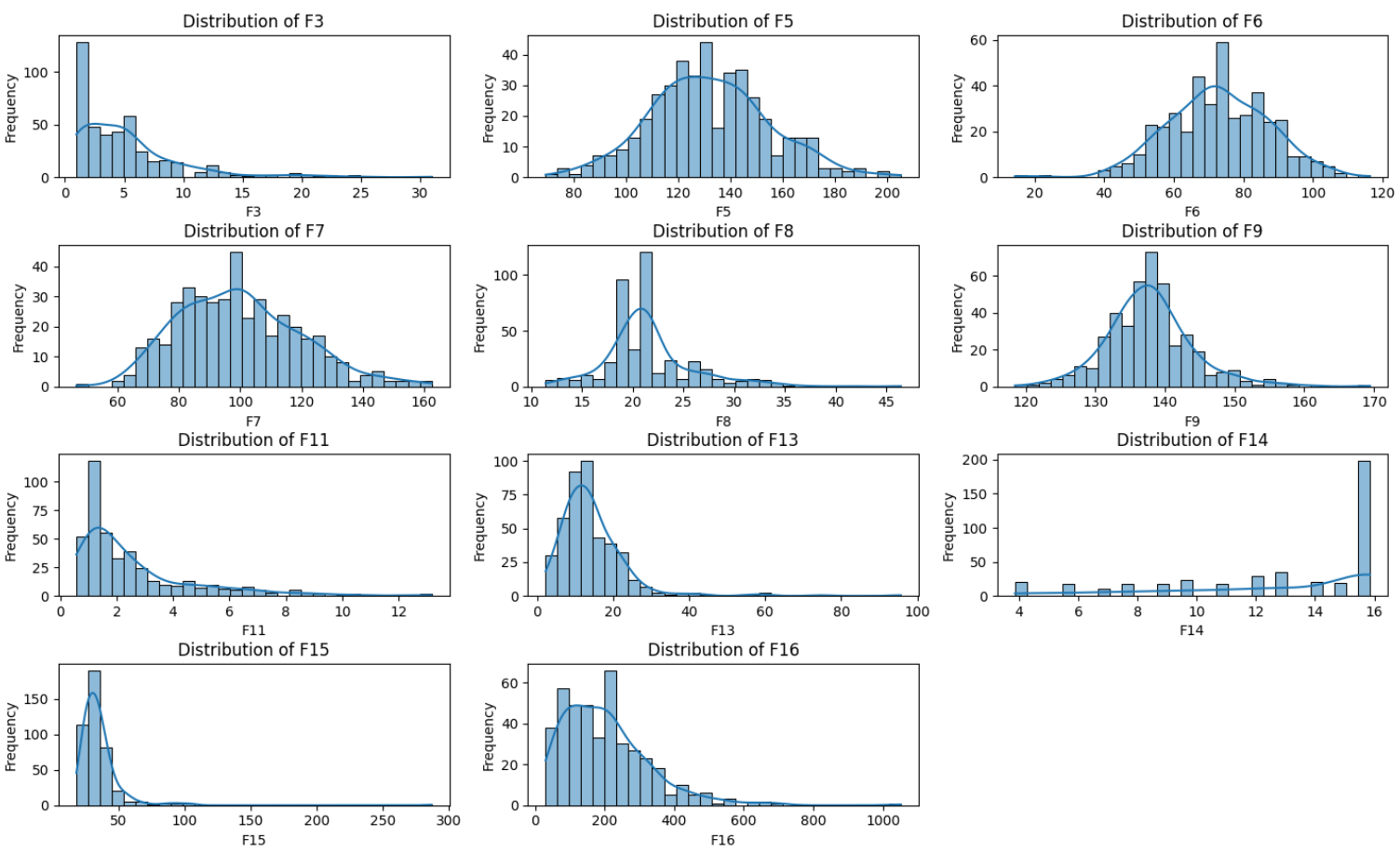
The provided `Naive Bayes Classifier` supports three modes:

1. **Continuous**: Handles numerical features using Gaussian distribution.
2. **Discrete**: Handles categorical features with probability tables.
3. **Mixed**: Combines both continuous and categorical features.

Workflow for the Naive Bayes Classifier Project

1. Feature Analysis and Preprocessing

1. **Analyze Feature Distribution**:
   - Start by analyzing the distributions of the continuous features in the dataset.
   - Use the `plot_continuous_distribution` function to visualize each feature's distribution and check for Gaussian-like behavior.



   - Document findings on whether the data aligns with the Gaussian assumption required for continuous features in Naive Bayes.
2. **Preprocess Features**:
   - Apply a preprocessing method similar to the one used in previous homework to boolize the categorical features (`F18-F77`).
     - Convert multi-class categorical features into binary (boolean) representations using one-hot encoding or similar techniques.
   - Use the `feature_selection` function to compute correlations with the label for feature selection.
     - Retain only features with a correlation above the threshold (e.g., > 0.1) to reduce non-informative features.

2. Initial Model Development

1. **Mixed-Mode Naive Bayes Classifier**:
   - Combine both preprocessed continuous and categorical features.
   - Train the Naive Bayes classifier in mixed mode, where Gaussian likelihoods are used for continuous features, and frequency-based likelihoods are used for categorical features.

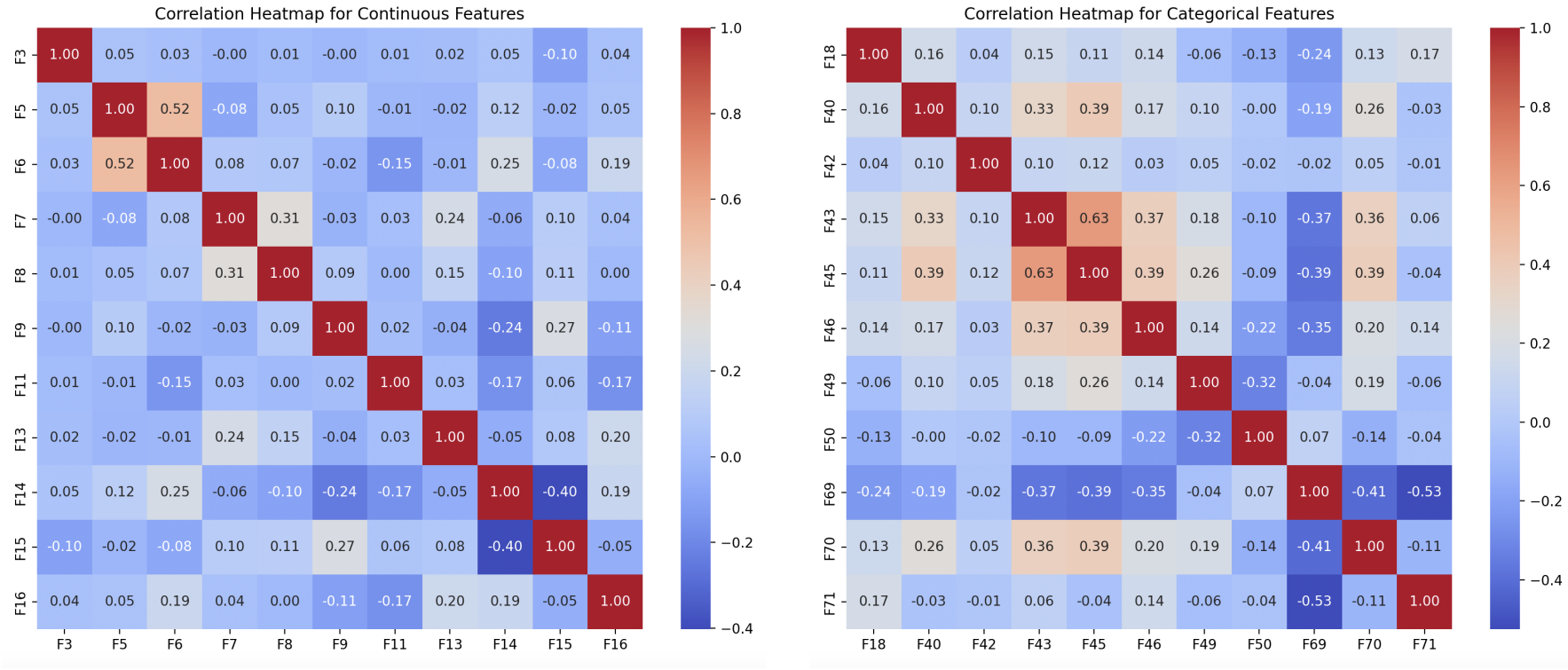
- Evaluate the model using K-Fold cross-validation to measure performance.

2. **Results of Mixed-Mode**:
   - The model only predicts 0.

---

3. Refinement and Discovery

1. **Investigate Categorical Feature Dependencies**:
   - Discover that many categorical features are dependent, violating Naive Bayes' assumption of feature independence.
   - Identify this dependency as the primary cause of poor performance in the mixed-mode classifier.



---

4. Transition to Continuous Mode

1. **Focus on Continuous Features**:
   - Modify the approach to use only the continuous features for training the Naive Bayes classifier.
   - Train the classifier in continuous mode, leveraging the Gaussian likelihood assumption.
2. **Evaluate the Continuous-Mode Model**:
   - Perform K-Fold cross-validation to validate the model's performance.
   - Observe significant improvement in scoring compared to the mixed-mode classifier.

---

5. Performance & Conclusion

- The final model using only continuous features provides the best performance.

```
k: 5
100%|                                                                                | 5/5 [00:00<00:00, 82.32it/s]
Kfold scoring: 0.5723445336838557
```

- This success highlights the importance of analyzing feature dependencies and aligning the model's assumptions with the data characteristics.

Key Code Details

1. Class Initialization

```
class NaiveBayesClassifier(Classifier):
    def __init__(self, mode='continuous'):
        self.mode = mode  # 'continuous', 'discrete', or 'mixed'
        self.priors = {}  # P(Class)
        self.classes = None
        self.features_mean = {}  # Mean values for continuous features
        self.features_var = {}   # Variance for continuous features
        self.categorical_prob = {}  # Probabilities for categorical features
        self.continuous_features = None
        self.categorical_features = None
```

- **Attributes**:
  - `mode`: Determines the type of features used:

    - `'continuous'`: Only continuous features are used.
    - `'discrete'`: Only categorical features are used.
    - `'mixed'`: Both continuous and categorical features are used.
  - `priors`: Stores the prior probabilities P(Class).

    P(Class)
  - `features_mean` and `features_var`: Store class-specific mean and variance for continuous features.
  - `categorical_prob`: Stores conditional probabilities for categorical features.
  - `continuous_features` and `categorical_features`: Separate the features into two groups for processing.

---

2. Fit Method

The `fit` method trains the Naive Bayes classifier using the input feature matrix X and target labels y.

```
def fit(self, X, y):
        """
        Fit the Naive Bayes model

        Parameters:
        -----------
        X : pandas DataFrame
            The input features
        y : array-like
            Target values
        """
        # Select features
        self.continuous_features = [col for col in X.columns if col.startswith('F') and 1 <= int(col[1:]) <= 17]
        self.categorical_features = [col for col in X.columns if col.startswith('F') and 18 <= int(col[1:]) <= 77]

        # Split features
        X_continuous = X[self.continuous_features]
        X_categorical = X[self.categorical_features]

        self.classes = np.unique(y)
        self.priors = self.calculate_prior(y)

        # Calculate likelihood parameters for continuous features
        if(self.mode == 'continuous' or self.mode == 'mixed'):
            self.features_mean, self.features_var = self.calculate_continuous_likelihood(X_continuous, y)

        # Calculate likelihood probabilities for categorical features
        if(self.mode == 'discrete' or self.mode == 'mixed'):
            self.categorical_prob = self.calculate_categorical_likelihood(X_categorical, y)
```

- **Key Steps**:
  1. **Feature Separation**:
     - Columns F1 to F17 are treated as continuous features.
     - Columns F18 to F77 are treated as categorical features.
     - This allows different handling for each type of feature.
  2. **Class and Prior Calculation**:
     - Identifies unique class labels.
     - Calculates prior probabilities $P(Class)$ for each class based on their frequency in y.
  3. **Likelihood Calculation for Continuous Features**:
     - For each continuous feature and class, calculates:
       - Mean (μ): Average feature value for the class.
       - Variance $(\sigma 2)$: Spread of feature values for the class.
     - These parameters are used in the Gaussian likelihood calculation.
  4. **Likelihood Calculation for Categorical Features**:
     - For each categorical feature and class, calculates:
       - Conditional probabilities $P(Feature \mid Class)$.

3. Prior Calculation:

- `calculate_prior(y)` computes the class probabilities $P(C)$ by dividing the count of each class by the total number of samples.

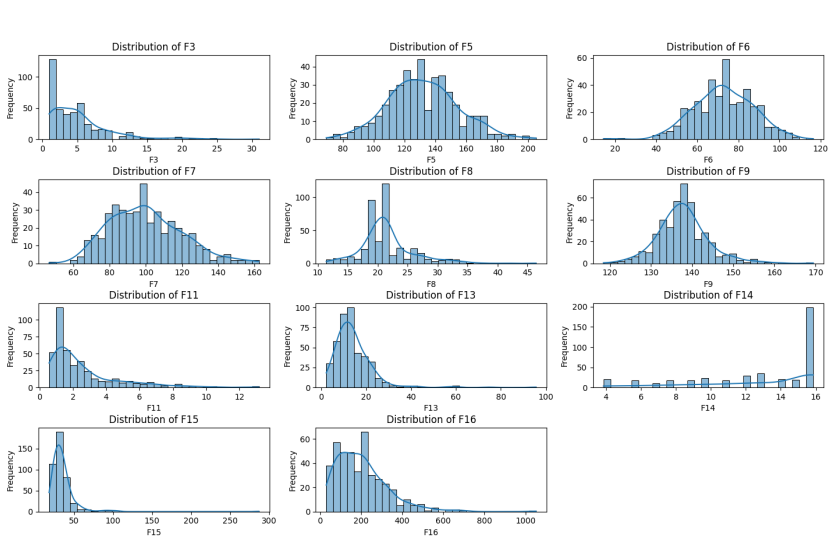
```
def calculate_prior(self, y):
    """Calculate prior probabilities P(Class) for each class"""
    total_samples = len(y)
    unique_classes, class_counts = np.unique(y, return_counts=True)
    return dict(zip(unique_classes, class_counts / total_samples))
```

4. Analysis and Visualization for Data Distribution:

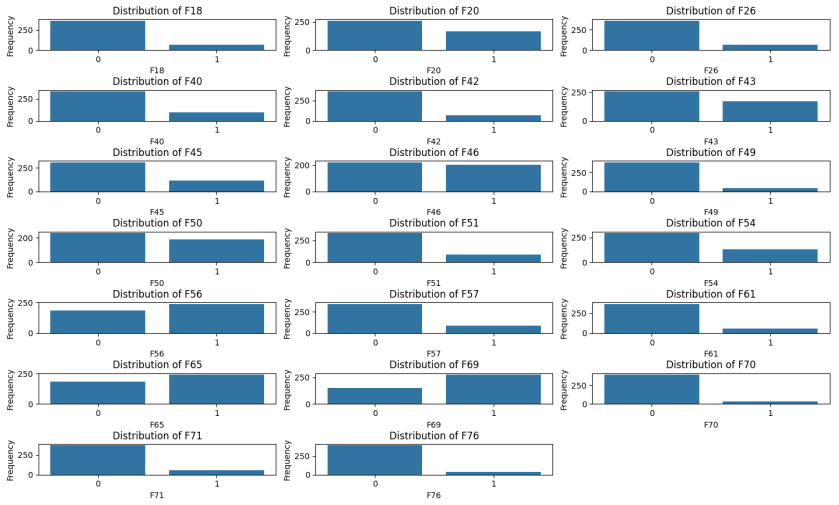
- **Feature Distribution (`plot_continuous_distribution` and `plot_categorical_distribution`)**:
  - Visualizes the distributions of continuous and categorical features.

```
def plot_continuous_distribution(self,X):
    # Plot the distribution for each continuous feature in one figure
    plt.figure(figsize=(15, 10))
    for i, feature in enumerate(self.continuous_features):
        plt.subplot(len(self.continuous_features)
        sns.histplot(X[feature], kde=True, bins=30)
        plt.title(f'Distribution of {feature}')
        plt.xlabel(feature)
        plt.ylabel('Frequency')
    plt.tight_layout()
    plt.show()
def plot_categorical_distribution(self,X):
    # Plot the distribution for each categorical feature in one figure
    plt.figure(figsize=(15, 10))
    for i, feature in enumerate(self.categorical_features):
        plt.subplot(len(self.categorical_features)
        sns.countplot(data=X, x=feature)
        plt.title(f'Distribution of {feature}')
        plt.xlabel(feature)
        plt.ylabel('Frequency')
    plt.tight_layout()
    plt.show()
```

- Continuous features are plotted using histograms with kernel density estimation (KDE).



- Categorical features are plotted using count plots.

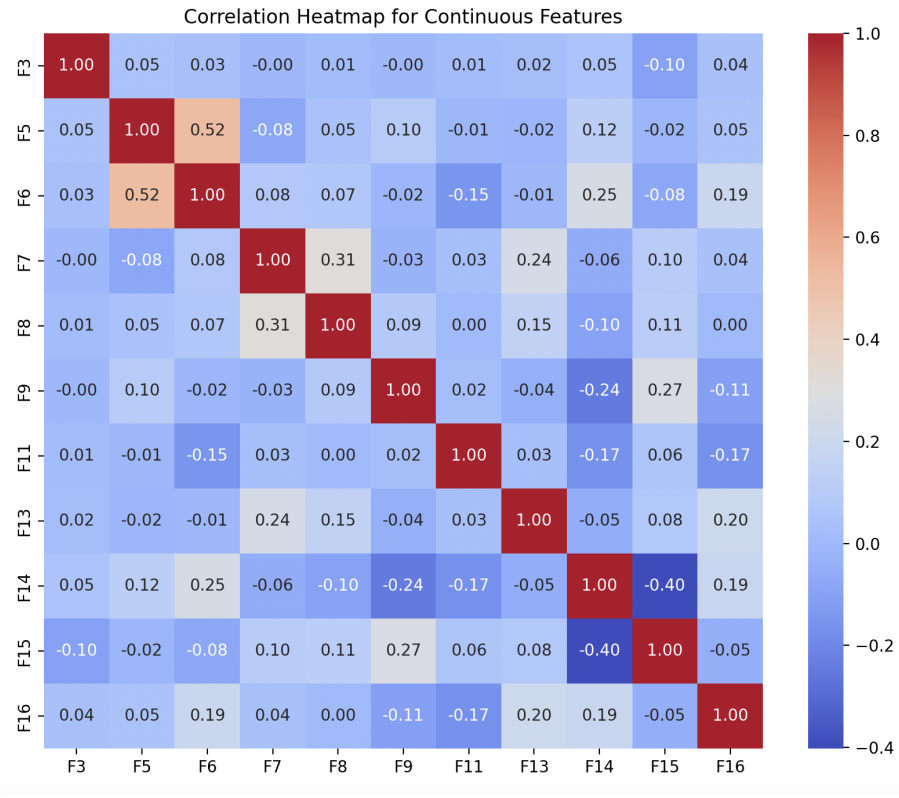


- **Feature Analysis (`categorical_analyze` and `continuous_analyze`)**:
  - Computes correlation matrices for categorical and continuous features separately.

```
def plot_continuous_distribution(self,X):
    # Plot the distribution for each continuous feature in one figure
    plt.figure(figsize=(15, 10))
    for i, feature in enumerate(self.continuous_features):
        plt.subplot(len(self.continuous_features) // 3 + 1, 3, i + 1)
        sns.histplot(X[feature], kde=True, bins=30)
        plt.title(f'Distribution of {feature}')
        plt.xlabel(feature)
        plt.ylabel('Frequency')
    plt.tight_layout()
    plt.show()
def plot_categorical_distribution(self,X):
    # Plot the distribution for each categorical feature in one figure
    plt.figure(figsize=(15, 10))
    for i, feature in enumerate(self.categorical_features):
```

```
        plt.subplot(len(self.categorical_features) // 3 + 1, 3, i + 1)
        sns.countplot(data=X, x=feature)
        plt.title(f'Distribution of {feature}')
        plt.xlabel(feature)
        plt.ylabel('Frequency')
    plt.tight_layout()
    plt.show()
```

- Visualizes the correlation matrices as heatmaps.



5. Likelihood Calculation:

- **Continuous Features**:
  - `log_likelihood` Function
    - `log_likelihood` function calculates the logarithm of the Gaussian likelihood for a given feature value x, given a class-specific mean (μ) and variance (σ^2).This is an alternative to directly computing the Gaussian likelihood, as done in the `gaussian_likelihood` function. Using logarithms improves numerical stability and avoids underflow issues that can arise when multiplying many small probabilities together.
    - **Why Use Log-Likelihood?**
      - **Numerical Stability**:
        - When probabilities are very small (e.g., \(10^{-10} \)), multiplying them can lead to values too small to be represented in typical floating-point formats.
        - Taking the logarithm transforms these small values into manageable negative numbers.
      - **Simplified Computation**:
        - Multiplying probabilities in the likelihood becomes summing logarithms in the log-likelihood:

$$\log(P(A \cdot B)) = \log(P(A)) + \log(P(B))$$

Mathematical Derivation

The Gaussian likelihood for a single feature is:

$$P(x|\mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

Taking the logarithm:

$$\log(P(x|\mu, \sigma^2)) = \log\left(\frac{1}{\sqrt{2\pi\sigma^2}}\right) + \log\left(\exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)\right)$$

Simplify each term:

1. Logarithm of the normalization term:

$$\log\left(\frac{1}{\sqrt{2\pi\sigma^2}}\right) = -\frac{1}{2}\log(2\pi\sigma^2)$$

2. Logarithm of the exponential term:

$$\log\left(\exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)\right) = -\frac{(x-\mu)^2}{2\sigma^2}$$

Combine these:

$$\log(P(x|\mu,\sigma^2)) = -\frac{1}{2}\log(2\pi\sigma^2) - \frac{(x-\mu)^2}{2\sigma^2}$$

```
def gaussian_likelihood(self, x, mean, var):
        """Calculate Gaussian likelihood P(Feature|Class) for continuous features"""
        exponent = np.exp(-((x - mean) ** 2) / (2 * var))
        return (1 / np.sqrt(2 * np.pi * var)) * exponent
def log_likelihood(self, x, mean, var):
    return -0.5 * np.log(2 * np.pi * var) - 0.5 * ((x - mean) ** 2) / var

def calculate_continuous_likelihood(self, X_continuous, y):
        """Calculate likelihood parameters for continuous features"""
        features_mean = {}
        features_var = {}

        for c in self.classes:
            X_class = X_continuous[y.iloc[:, -1] == c]
            if not X_class.empty:  # If there are continuous features
                features_mean[c] = X_class.mean()
                features_var[c] = X_class.var()
            else:
                features_mean[c] = pd.Series([])
                features_var[c] = pd.Series([])

        return features_mean, features_var
```

- **Categorical Features**:
  - Uses frequency counts with Laplace smoothing to avoid zero probabilities:

$$P(X_i|C) = \frac{\text{Count}(X_i|C) + 1}{\text{Total Count}(C) + k}$$

    where k is the number of unique values for the feature.

```
def calculate_categorical_likelihood(self, X_categorical, y):
        """Calculate likelihood probabilities for categorical features"""
        categorical_prob = {}
        for c in self.classes:
            categorical_prob[c] = {}
            X_class = X_categorical[y.iloc[:, -1] == c]
            #print(X_class)
            for col in X_categorical.columns:
                # Calculate probability of each category given the class
                value_counts = X_class[col].value_counts()
                # Add Laplace smoothing
                smoothed_counts = value_counts + 1
                probs = smoothed_counts / smoothed_counts.sum()
                categorical_prob[c][col] = probs.to_dict()

        return categorical_prob
```

6. Prediction:

- `predict_proba(X)` calculates the log probabilities of each class for a given input using:
  - Prior probabilities.
  - Gaussian likelihood for continuous features.
  - Probability tables for categorical features.
- `predict(X)` determines the class with the highest posterior probability.

```
def predict_proba(self, X):
        """
        Predict class probabilities for samples

        Parameters:
```

```
        -----------
        X : pandas DataFrame
            The input features
        """
        # Split features using saved feature lists
        X_continuous = X[self.continuous_features]
        X_categorical = X[self.categorical_features]

        probas = np.zeros((len(X), len(self.classes)))

        for i, c in enumerate(self.classes):
            # Calculate prior probability
            class_prob = np.log(self.priors[c])

            # Calculate likelihood for continuous features
            if self.mode == 'continuous' or self.mode == 'mixed':
                for feature in self.continuous_features:
                    x = X_continuous[feature]
                    mean = self.features_mean[c][feature]
                    var = self.features_var[c][feature]
                    class_prob += self.log_likelihood(x, mean, var)
            if self.mode == 'discrete' or self.mode == 'mixed':
                # Calculate likelihood for categorical features
                for feature in self.categorical_features:
                    for idx in range(len(X)):
                        category = X_categorical.iloc[idx][feature]
                        # If category wasn't seen during training, use Laplace smoothing probability
                        prob = self.categorical_prob[c][feature].get(category, 1.0 / len(self.categorical_prob[c][fea

                        class_prob += np.log(prob)

            probas[:, i] = class_prob
        log_prob_max = np.max(probas, axis=1, keepdims=True)
        probs = np.exp(probas - log_prob_max)
        probs = probs / np.sum(probs, axis=1, keepdims=True)

        return probas

def predict(self, X):
        """Predict class labels for samples"""
        probas = self.predict_proba(X)
        return self.classes[np.argmax(probas, axis=1)]
```

7. K-Fold cross validation:

- Splits the dataset into n folds.
- Iteratively uses one fold for validation and the remaining n−1 folds for training.
- Evaluates the model using three metrics: **Accuracy**, **F1-Score**, and **Matthews Correlation Coefficient (MCC)**.
- Returns the averaged metrics and a combined scoring value.

```
def KFold_cross_validation(X, y, n, k=5):
    fold_size = len(X) // n
    data = pd.concat([X, y], axis=1)
    data = data.sample(frac=1).reset_index(drop=True)

    scores = {
        'accuracy': [],
        'f1': [],
        'mcc': [],
    }
    print('k:', k)
    for i in tqdm(range(n)):
        start, end = i * fold_size, (i + 1) * fold_size
        val_data = data[start:end]
        train_data = pd.concat([data[:start], data[end:]])

        X_train, y_train = train_data.iloc[:,:-1], pd.DataFrame(train_data.iloc[:,-1])
        X_val, y_val = val_data.iloc[:,:-1], val_data.iloc[:,-1]
```

```
        model = NaiveBayesClassifier(mode='continuous')
        model.fit(X_train, y_train)
        y_pred = model.predict(X_val)
        scores['accuracy'].append(accuracy_score(y_val, y_pred))
        scores['f1'].append(f1_score(y_val, y_pred))
        scores['mcc'].append(matthews_corrcoef(y_val, y_pred))

    for metric in scores:
        scores[metric] = np.mean(scores[metric])
    scoring = 0.3 * scores['accuracy'] + 0.35 * scores['f1'] + 0.35 * scores['mcc']
    scores['scoring'] = scoring
    print('Kfold scoring:', scoring)
    print('\n')
    return scores
```

Questions

(a) Fixing Zero Probability in Bayesian Prediction

Why is it necessary to fix zero probability?

Zero probabilities occur when a feature value is not observed for a class during training, leading to $P(X|C) = 0$. This results in zero posterior probability $P(C|X)$ for that class, even if the class is otherwise likely.

Solution:

Use Laplace smoothing, which adds a pseudo-count to each feature value:

$$P(X_i|C) = \frac{\text{Count}(X_i|C) + 1}{\text{Total Count}(C) + k}$$

Example:

Consider a spam classifier with the feature "word='offer'" and $P(\text{offer}|\text{not spam}) = 0$ because "offer" was not observed in "not spam" emails.

Without smoothing, any email containing "offer" will always be classified as spam, which is incorrect.

(b) Handling Dependent Features in Naïve Bayes

Naïve Bayes assumes feature independence, which may not hold in real-world data. When features are dependent:

- **Problem**: The independence assumption leads to incorrect joint likelihoods.

Solution:

Modify Naïve Bayes to account for dependencies:

1. **Tree-Augmented Naïve Bayes (TAN)**:
   - Constructs a dependency tree between features and incorporates conditional probabilities.
2. **Feature Selection**:
   - Remove highly correlated features using methods like mutual information or correlation thresholds.
   - And this is the solution I used in this project for Categorical Features.
3. **Bayesian Networks**:
   - Explicitly model dependencies between features using a directed acyclic graph (DAG).

Reason:

TAN and Bayesian Networks relax the independence assumption, providing more accurate predictions while retaining Bayesian principles.

Discussion

(a) Distributions and Bayesian Methods

Bayesian methods rely on prior and likelihood distributions. The choice of distribution affects the model's performance and applicability:

1. **Gaussian Distribution**:
   - Used for continuous features assuming a bell-shaped distribution.
   - Common in real-world applications due to its simplicity and mathematical properties.
2. **Uniform Distribution**:
   - Assumes equal probability for all outcomes.
   - Suitable for features with limited knowledge or equal likelihood.
3. **Poisson Distribution**:

- Models count data, e.g., the number of occurrences in a fixed interval.
- Often used in event prediction tasks.

Relationship:

The likelihood function $P(X|C)$ is defined by the chosen distribution.

For example, Gaussian likelihood is appropriate for continuous features, while Poisson is suitable for discrete count data.

But you should use visualize the data distribution to find the best choice for your bayes model.

(b) Problem Encountered in Building Model

1. **Failure of Mixed-Mode Approach**:
   - The poor performance of the mixed-mode Naive Bayes classifier was attributed to the high dependency among categorical features. This violates Naive Bayes' core assumption that features are conditionally independent given the label.
2. **Solution and Transition to Continuous Mode**:
   - By excluding the categorical features and focusing solely on continuous features, the model's assumptions were better aligned with the data.
   - The resulting continuous-mode Naive Bayes classifier achieved significantly improved performance, demonstrating the importance of matching the model's assumptions to the dataset characteristics.
